COMP202 Data Structures and Algorithms
Assignment 2 — Bonus Part: HMAC Guide

Instructor: Alptekin Küpçü
Prepared by: Halil İbrahim Kanpak

Important Note

This document provides the background material needed to complete the **optional bonus part** of Assignment 2. The bonus part is worth 20 points, but your total assignment grade is capped at 100. You do **not** need to complete this part to receive full marks on the assignment. Read this guide carefully before attempting `HashCrypto.java`.

1. Motivation: Why Do We Need Message Authentication?

Imagine Alice sends a message to Bob over an insecure network. An attacker (Eve) sitting in the middle could intercept the message and modify it before it reaches Bob. How can Bob be sure that the message he received is exactly what Alice sent?

One solution is to attach a short "tag" or "code" to the message. This code is computed using the message contents and a **secret key** that only Alice and Bob know. When Bob receives the message, he recomputes the code using the same secret key. If his computed code matches the one Alice attached, he can be confident that:

1. The message was not modified in transit (**integrity**).
2. The message came from someone who knows the secret key (**authenticity**).

This code is called a **Message Authentication Code (MAC)**.

2. Hash Functions Revisited

In Part 2 of this assignment, you implemented a **polynomial rolling hash function** to map string keys to bucket indices in a hash table. The formula was:

$$h(s) = \left(s_0 \cdot 31^{n-1} + s_1 \cdot 31^{n-2} + \cdots + s_{n-1}\right) \bmod \text{capacity}$$

where s_i is the integer value of the character at position i, and n is the length of the string.

In the hash table, the modulus was the table's capacity (e.g., 8, 16, 32, ...). For the bonus part, we will use the same formula but with a **prime number** as the modulus instead. Using a prime modulus (we use 10007) produces a more uniform distribution of hash values, which is desirable for cryptographic purposes.

Key Concept

The `hashMessage(String msg, int mod)` method is **exactly the same** as your `hash(String key)` method from Part 2, except it takes the modulus as a parameter instead of using the table's capacity. If you implemented `hash()` correctly, you can reuse the same logic.

3. What is HMAC?

HMAC stands for **Hash-based Message Authentication Code**. In real-world cryptography (RFC 2104), HMAC uses a secure hash function like SHA-256 and involves XOR operations with padded keys. Our

version is a **simplified** HMAC that uses the polynomial rolling hash instead. The core idea is the same: combine a secret key with the message through hashing to produce an authentication code.

3.1 Our Simplified HMAC — Step by Step

Given a message `msg` and a secret key `key`, the simplified HMAC is computed in **two steps**:

HMAC Algorithm

Step 1 — Inner Hash:

1. Concatenate the secret key and the message: `innerInput = key + msg`
2. Compute the hash: `innerHash = hashMessage(innerInput, 10007)`

Step 2 — Outer Hash:

1. Convert `innerHash` (an integer) to a string: `String.valueOf(innerHash)`
2. Concatenate the secret key and the string representation of `innerHash`: `outerInput = key + String.valueOf(innerHash)`
3. Compute the hash: `outerHash = hashMessage(outerInput, 10007)`

Result: The HMAC code is `outerHash`.

3.2 Why Two Steps?

The two-step structure is important for security. A single hash of `key + msg` is vulnerable to a class of attacks called **length-extension attacks**, where an attacker who knows $h(\text{key} + \text{msg})$ can compute $h(\text{key} + \text{msg} + \text{extra})$ without knowing the key. The second hashing step (the "outer" hash) prevents this by wrapping the result in another hash computation that includes the key again.

While our simplified version does not provide real cryptographic security (our hash function is too simple), it illustrates the principle behind the real HMAC construction.

4. Worked Example

Let us walk through a complete example step by step.

Given:

- Message: `"hello"`
- Secret key: `"key"`
- Prime modulus: `10007`

Step 1: Compute innerHash

Concatenate key and message: `"key" + "hello" = "keyhello"`

Now compute `hashMessage("keyhello", 10007)`:

The characters and their integer values are:

Index i	0	1	2	3	4	5	6	7
Character	k	e	y	h	e	l	l	o
ASCII value s_i	107	101	121	104	101	108	108	111

Using the polynomial rolling hash (computed iteratively with `long` arithmetic):

$$\begin{aligned}
&\text{hash} = 0 \\
&\text{hash} = 0 \times 31 + 107 = 107 \\
&\text{hash} = 107 \times 31 + 101 = 3418 \\
&\text{hash} = 3418 \times 31 + 121 = 106079 \\
&\text{hash} = 106079 \times 31 + 104 = 3288553 \\
&\text{hash} = 3288553 \times 31 + 101 = 101945244 \\
&\text{hash} = 101945244 \times 31 + 108 = 3160302672 \\
&\text{hash} = 3160302672 \times 31 + 108 = 97969382940 \\
&\text{hash} = 97969382940 \times 31 + 111 = 3037050871251
\end{aligned}$$

Apply the modulus: $3037050871251 \bmod 10007 = 3037050871251 - \lfloor 3037050871251/10007 \rfloor \times 10007$

Computing: $3037050871251 \bmod 10007 = \mathbf{4}$ (the actual result depends on precise arithmetic; your Java code using `long` will compute this correctly).

Note: The exact numeric result above is for illustration. When you implement this in Java using `long` types, the computation will handle the large intermediate values correctly. What matters is that you apply the formula iteratively: `hash = hash * 31 + (int) c` for each character, and then take `((hash % mod) + mod) % mod` at the end.

Let us call the result `innerHash` (whatever integer your Java code produces).

Step 2: Compute outerHash

Convert `innerHash` to a string (e.g., if `innerHash = 4217`, the string is `"4217"`).

Concatenate: `"key" + "4217" = "key4217"`

Compute `hashMessage("key4217", 10007)` using the same procedure.

The result is the final HMAC code.

Verification

To verify: given the message `"hello"`, key `"key"`, and the HMAC code, Bob simply recomputes `generateHMAC("hello", "key")` and checks whether the result matches the received code.

5. Detecting Tampered Messages

The `detectTampering` method receives:

- An array of messages: `["hello", "world", "foo"]`
- An array of HMAC codes: `[4217, 9999, 8312]` (one code per message)
- A secret key: `"key"`

For each message, you verify its HMAC code using `verifyHMAC`. If the verification fails (the code does not match), that message has been **tampered with**. Collect all tampered messages into a result array, preserving their original order.

Example: If `"world"` was modified by an attacker, its HMAC code (9999) will not match the recomputed HMAC. So `"world"` would appear in the returned array.

Implementation approach (two-pass):

1. **First pass:** Iterate through all messages and count how many fail verification.
2. Create a `String[]` array of exactly that count.
3. **Second pass:** Iterate again and fill in the tampered messages, preserving their original order.

6. Summary of Methods

Method	Points	Description
`hashMessage(msg, mod)`	3	Same as `hash()` from Part 2 but with a custom modulus parameter.
`generateHMAC(msg, key)`	7	Two-step HMAC: inner hash of `key+msg`, then outer hash of `key+innerHash`.
`verifyHMAC(msg, key, code)`	3	Recompute HMAC and compare with the given code.
`detectTampering(msgs, codes, key)`	7	Return array of messages whose HMAC codes do not match (tampered).

7. Tips

- `hashMessage` is almost identical to your `hash()` from Part 2. The only difference is the modulus parameter. If you got Part 2 working, this should be straightforward.
- For `generateHMAC`, pay attention to string concatenation: use `+` for strings, and `String.valueOf()` to convert the integer `innerHash` to a string before the second concatenation.
- For `detectTampering`, use the two-pass approach described above. This is the same pattern used in `keysWithValueAbove()` from Part 2.
- Remember to use `long` arithmetic in `hashMessage` to avoid integer overflow, just as you did in `hash()`.
- Test your implementation using the test cases provided in `Main.java`.